firstCV에 바로 쓸 수 있는 논문 3편 요약

아이폰 실시간 사물 인식 · 보행 보조 프로젝트용 정리 · 2026년 11월

한눈에 보기

firstCV의 현재 병목은 모델 크기가 아니라 **데이터**입니다. 선생님 모델이 만든 라벨에 노이즈가 많고, 장면도 한쪽에 치우쳐 있습니다. 아래 세 편은 각각 이 문제의 한 부분을 다룹니다.

논문	다루는 문제	firstCV에 주는 것
① Per-Object Co-teaching arXiv 2511.09955	선생님(VLM) 라벨의 노이즈를 **학습 단계**에서 걸러 내는 방법	학습 코드에 붙일 구체적인 알고리즘. 사람 검수 라벨 10%의 효과를 보여 주는 수치
② Auto-Labeling Data for Object Detection arXiv 2506.02359	선생님 모델로 **라벨을 만드는 단계**의 설정값(모델, 신뢰도 기준)	445번의 실험에서 나온 라벨링 설정 지침. 현재 `--conf 0.2` 가 맞는 선택이라는 근거
③ PEDESTRIAN 데이터셋 arXiv 2512.19190	휴대폰으로 찍은 **보행자 시점 보도 장애물** 영상	보행 장면 영상(박스 라벨 없음). 우리 선생님 파이프라인에 넣을 원본 데이터 후보

> **앞선 대화의 설명을 바로잡습니다**
>
> 대화에서는 ③ PEDESTRIAN을 "장애물 29종이 라벨링된 탐지 데이터셋"이라고 소개했습니다. 원문을 읽어 보니 **영상마다 장애물 이름 하나만 붙은 이미지 분류용 데이터**였고, 박스 좌표는 없습니다. 그래서 탐지 학습에 바로 쓸 수는 없고, 선생님 모델로 라벨을 새로 만들어야 합니다. 자세한 내용은 ③ 요약에 정리했습니다.

추천 실행 순서

- **1단계 (거의 비용 없음):** ②의 지침대로 라벨링 설정을 점검합니다. 신뢰도 기준은 0.2로 유지하고, 성적이 낮은 book·chair는 프롬프트 문구를 더 구체적으로 바꿔 봅니다.
- **2단계 (가장 큰 효과 예상):** 검증 세트 일부를 사람이 검수해 정답 라벨로 만듭니다. ①에서는 정답 라벨을 10% 섞자 mAP50이 46.6%에서 58.0%로 올랐습니다. 이 작업은 평가 신뢰도 문제도 함께 풀어 줍니다.
- **3단계 (구현이 필요함):** ①의 객체 단위 co-teaching을 학습 코드에 붙이고, 같은 데이터에서 기존 학습과 비교합니다.
- **병행:** ③의 영상을 받아 라이선스를 확인한 뒤, 보행 장면용 라벨링 대상 원본으로 씁니다.

논문 ① · 학습 단계의 노이즈 대응

Robust Object Detection with Pseudo Labels from VLMs using Per-Object Co-teaching

Bhaskar, Bhattacharya 외 (IIIT Hyderabad, Bosch) · arXiv 2025.11 · arxiv.org/abs/2511.09955

> **VLM이 만든 노이즈 라벨로 YOLO를 학습시킬 때, YOLO 두 개가 서로 "믿을 만한 박스"를 골라 주게 하면 그냥 학습할 때보다 mAP50이 크게 오른다.**

풀려는 문제

VLM(이 논문에서는 OWLv2)은 텍스트 프롬프트만으로 물체를 찾아 주지만, 느려서 실시간으로 돌릴 수 없고 헛것(hallucination)도 자주 만듭니다. 그래서 VLM으로 라벨을 만들고 작은 YOLO를 학습시키는 방식이 쓰입니다. 그런데 라벨에 섞인 틀린 박스와 틀린 클래스를 YOLO가 그대로 외워 버리는 문제가 있습니다. firstCV의 "선생님 → 학생" 구조와 똑같은 상황입니다.

방법

- **Co-teaching의 아이디어:** 신경망은 깨끗하고 쉬운 패턴을 먼저 배우고 노이즈는 나중에 외웁니다. 그래서 학습 초중반에 loss가 작은 샘플은 정답일 가능성이 높습니다. 초기값이 다른 두 모델이 각자 loss가 작은 샘플을 골라 **상대 모델**에게 넘겨 학습시킵니다. 한 모델의 실수가 자기 자신에게 되먹임되지 않는 구조입니다.
- **이 논문의 핵심, 객체 단위 선택:** 기존 co-teaching은 이미지 단위로 버립니다. 하지만 탐지에서는 한 이미지 안에 맞는 박스와 틀린 박스가 섞여 있어서, 이미지째 버리면 멀쩡한 라벨까지 잃습니다. 그래서 **박스(positive anchor) 단위로** loss를 비교해 하위 k개만 남깁니다.
- **선택 기준 loss:** `ℓ_sel = ℓ_box + ℓ_cls` 입니다. objectness loss는 배경에 민감해서 기준에서 뺐습니다(아래 실험으로 검증).
- **버리는 비율을 점점 늘리기:** 처음에는 거의 다 믿다가 비율 r을 선형으로 키웁니다. 논문은 150 epoch 동안 0에서 0.2까지 올렸습니다. 학습이 끝나면 두 모델 중 아무거나 하나를 배포하면 됩니다.

핵심 결과

방법 (YOLOv5m)	KITTI	ACDC (악천후)	BDD100K
OWLv2 직접 추론 (선생님)	32.3	18.8	30.8
의사 라벨로 그냥 학습 (기준선)	31.1	20.1	32.1
Soft distillation	34.1	21.8	35.1
Data distillation	37.2	25.4	37.4
이미지 단위 co-teaching	39.4	23.1	36.9
객체 단위 co-teaching (제안)	**46.6**	**27.3**	**43.2**
사람 정답 라벨로 학습 (상한)	90.3	29.6	51.9

mAP50 (%). 학생이 선생님(OWLv2)보다 높은 점수를 낸다는 점도 눈여겨볼 만합니다.

의사 라벨 : 정답 라벨	KITTI mAP50
100 : 0	46.6
95 : 5	49.4
90 : 10	**58.0**
85 : 15	65.1
80 : 20	72.4

선택 기준 loss	mAP50
box + cls	**46.6**
box + cls + obj	43.8
obj만	36.4
cls + obj	34.2
box만	31.5

75 : 25	77.8

정답 라벨을 조금만 섞어도 효과가 큽니다.

cls만	4.5

objectness를 빼는 것이 핵심입니다.

라벨이 없는 데이터를 60%에서 100%로 늘리자 mAP50도 38.3%에서 46.6%로 꾸준히 올랐습니다. 노이즈를 걸러 내기 때문에 데이터를 더 모을수록 이득이 쌓인다는 뜻입니다.

firstCV에 적용한다면

- **가장 먼저 할 일은 정답 라벨 10%입니다.** 검증용 99장을 사람이 검수하면 평가가 "선생님과의 일치도"에서 "실제 정답률"로 바뀝니다. 같은 작업으로 학습용 정답 데이터도 생깁니다.
- book(0.09), chair(0.15), phone(0.28)처럼 선생님 라벨이 흔들리는 클래스에서 효과가 클 것으로 예상합니다.
- 구현할 때는 Ultralytics 학습 루프의 loss 계산 부분을 바꿔서, 두 모델이 positive anchor별 `box + cls` loss를 주고받고 마스크를 씌워야 합니다. 앱에는 둘 중 하나만 배포하므로 추론 속도(16ms)는 그대로입니다.

주의할 점

- 논문은 YOLOv5m을 **사전학습 없이 처음부터** 300 epoch 학습했습니다. 우리는 사전학습된 YOLO26s를 미세조정하므로, 기준선이 이미 높아서 상승 폭이 논문보다 작을 수 있습니다.
- 모델 두 개를 동시에 올리므로 학습 메모리가 2배로 듭니다. 맥(MPS)에서는 배치 크기를 줄여야 할 수 있습니다.
- 버리는 비율의 최댓값(논문 0.2)은 라벨 노이즈 비율의 추정치입니다. 우리 데이터에 맞는 값은 사람 검수 결과로 따로 추정해야 합니다.
- KITTI처럼 쉬운 데이터에서는 정답 라벨로 학습한 상한(90.3)과 차이가 여전히 큽니다. Co-teaching이 사람 검수를 대체하지는 못합니다.

논문 ② · 라벨 생성 단계의 설정

Auto-Labeling Data for Object Detection

Griffin, Gangwar, Sela, Corso (Voxel51, University of Michigan) · arXiv 2025.06 · arxiv.org/abs/2506.02359

> **선생님 모델의 라벨로 작은 탐지기를 학습시킬 때는 신뢰도 기준을 낮게(약 0.2) 잡아야 한다. 정밀도가 가장 높은 설정이 오히려 최악의 학생을 만든다.**

풀려는 문제

사람 라벨 없이 VLM이 만든 라벨만으로 경량 탐지기를 학습시키는 "오토 라벨링"이 실제로 쓸 만한지, 쓴다면 어떤 설정이 좋은지를 대규모 실험으로 확인했습니다. 선생님 모델 3종(YOLO-World, **YOLOE**, Grounding DINO), 학생 모델 6종(YOLO11 n·s·m·l·x, RT-DETR), 데이터셋 4종(VOC, COCO, LVIS, BDD)을 조합해 학습 실험을 445번 돌렸습니다. 우리 파이프라인이 YOLOE를 선생님으로 쓰기 때문에 결과를 그대로 참고할 수 있습니다.

핵심 발견

- **신뢰도 기준은 낮게:** 학생 성능이 가장 좋은 지점은 라벨 F1이 가장 높은 지점이 아니라 **재현율(recall)이 높은 쪽**이었습니다. α = 0.1~0.2 부근입니다. 반대로 α = 0.8은 라벨 정밀도는 가장 높았지만 모든 조합에서 학생 성능이 가장 나빴습니다. 박스를 놓치는 것이 틀린 박스보다 더 해롭다는 뜻입니다.
- **가장 안정적인 설정:** YOLO-World α = 0.2가 네 데이터셋 모두에서 안정적이었습니다. YOLOE도 α = 0.2에서 항상 자기 최고 성능을 냈습니다. Grounding DINO는 α에 매우 민감했습니다.
- **학생 크기가 라벨 출처보다 중요:** 같은 크기의 모델끼리는 사람 라벨이 약간 낫습니다. 하지만 "YOLO11n + 사람 라벨"보다 "YOLO11s + 오토 라벨"이 더 높았습니다(VOC 0.756 → 0.768, COCO 0.496 → 0.538).
- **도메인이 낯설면 격차가 커짐:** 주행 시점 영상인 BDD에서 사람 라벨과의 격차가 가장 컸습니다(0.434 대 0.271). 선생님 모델이 학습한 데이터와 시점이 달라서 라벨 품질이 떨어졌기 때문입니다.
- **이름이 모호한 클래스는 아예 라벨이 안 생김:** BDD의 "rider", "motor" 같은 클래스는 어떤 α에서도 라벨이 거의 생기지 않았습니다. 논문은 더 구체적인 프롬프트 문구를 해결책으로 제시합니다.

YOLO11n 학습 라벨	VOC	COCO	LVIS	BDD	라벨링 비용·시간
사람 라벨	0.756	0.496	0.087	0.434	약 $124,000 · 6,703시간
YOLO-World α=0.2	**0.715**	**0.460**	**0.059**	**0.271**	**$1.18 · 1.3시간**

mAP50. 평균적으로 사람 라벨 성능의 약 85%를 비용 10만분의 1로 얻었습니다.

우리 약점과 겹치는 클래스별 결과 (COCO, YOLO11n, AP50)

클래스	오토 라벨 (α=0.2)	사람 라벨	firstCV 현재
book	0.06~0.09	0.179	0.09 (가장 낮음)
chair	0.11~0.33	0.359	0.15
bottle	0.21~0.41	0.411	0.54

범위는 선생님 모델 3종의 결과입니다. book은 사람 라벨로 학습해도 어려운 클래스이고, 오토 라벨에서는 성능이 절반 아래로 떨어집니다.

firstCV에 적용한다면

- **현재 설정이 맞습니다.** `scripts/04_auto_label.py`의 기본값 `--conf 0.2`는 이 논문이 YOLOE에 권하는 값과 같습니다. 오답을 줄이겠다고 0.3 이상으로 올리는 것은 이 논문 결과에 비추어 보면 손해일 가능성이 큽니다.
- **book·chair는 오토 라벨만으로는 한계가 있습니다.** 이 두 클래스는 우선적으로 사람 검수를 넣거나 프롬프트 문구를 바꿔 봅니다. 예를 들어 "book" 대신 "hardcover book", "open book"처럼 여러 문구를 쓰는 식입니다.

- **보행 모드는 BDD와 같은 위험이 있습니다.** 걸으면서 아래를 비추는 시점은 선생님 모델이 익숙한 웹 사진과 다릅니다. 보도 장면에서는 라벨 품질을 따로 확인해야 합니다.
- 정확도를 더 올려야 하는데 속도에 여유가 있다면(16ms), 라벨을 더 다듬는 것보다 학생 모델을 키우는 쪽이 효율적일 수 있습니다.

주의할 점

- 여기서도 학생 모델을 **사전학습 없이** 100 epoch 학습했습니다. 사전학습 모델을 미세조정하는 우리와는 조건이 다릅니다.
- SAM 3처럼 최신 선생님 모델은 비교 대상에 없습니다. α 권장값이 다른 모델에도 그대로 통한다는 보장은 없습니다.
- 라벨링 설정을 비교하는 실험 논문이라, 노이즈를 줄이는 학습 방법은 다루지 않습니다. 그 부분은 ①이 보완합니다.

논문 ③ · 보행 장면 데이터

PEDESTRIAN: An Egocentric Vision Dataset for Obstacle Detection on Pavements

Thoma, Theodosiou 외 (CYENS Centre of Excellence, 키프로스) · arXiv 2025.12 · arxiv.org/abs/2512.19190 · 데이터: Zenodo DOI 10.5281/zenodo.10907945

> **스마트폰을 가슴 높이에 들고 보도를 걸으며 찍은 장애물 영상 340개(29종). 다만 박스 라벨이 없는 분류용 데이터라서, 우리에게는 "라벨링할 원본 보행 영상"으로서 가치가 있다.**

데이터 구성

- **촬영:** 키프로스 니코시아의 보도. Xiaomi Mi Mix 3(60fps)와 iPhone 7(30fps) 두 기종으로, 가슴 높이에서 세로 1080×1920으로 찍었습니다. 낮·흐림·밤 세 가지 조명 조건이 있고, 얼굴과 번호판은 흐림 처리되어 있습니다.
- **라벨:** 영상 하나에 장애물 하나만 담기도록 찍고, **영상 단위로 장애물 이름**을 붙였습니다. 분류 체계는 대분류 → 중분류 → 장애물 종류의 3단계입니다. 박스나 분할 마스크는 없습니다.
- **29종 예시:** 2륜차, 4륜차, 벤치, 쓰레기통, 의자, 테이블, 라바콘, 가로수, 주차 방지 기둥, 가로등, 공사 비계, 구멍(포트홀), 균열, 깨진 보도블록, 좁은 보도, 보도 없음, 붐비는 보도 등
- **규모:** 영상 340개를 공개했고, 실험에는 종류마다 프레임 500장씩 뽑은 균형 세트(14,500장)를 썼습니다. 전체 용량은 5.9GB이며 GitHub에 로더 코드도 있습니다.

벤치마크 결과

ImageNet으로 사전학습한 분류 모델 16종(MobileNetV3-Small 2.5M부터 ConvNeXt-Large 198M까지)을 미세조정했습니다. 장애물 종류 분류에서 **모든 모델이 정확도 99% 이상**을 기록했고, 실험은 총 480회였습니다.

주의할 점: 99%라는 숫자는 그대로 믿기 어렵습니다

- **탐지가 아니라 분류입니다.** 제목에는 "Obstacle Detection"이 들어가지만, 실험은 이미지 한 장에 장애물 이름 하나를 맞히는 분류입니다. 탐지 모델의 mAP와는 비교할 수 없습니다.
- **데이터 누수 가능성:** 프레임을 영상 구분 없이 무작위로 70/20/10으로 나눴습니다. 같은 영상의 바로 옆 프레임이 학습과 시험에 함께 들어가므로 점수가 부풀었을 가능성이 큽니다. 논문은 이 점을 따로 다루지 않으며, 이 판단은 정리하면서 덧붙인 의견입니다. firstCV README가 "검증 세트는 학습에 쓰지 않은 별도 영상"이라고 밝힌 것이 이 점에서 더 엄격한 설계입니다.
- 한 도시에서만 찍었고, 한국 보도의 점자블록이나 볼라드 모양과는 다릅니다.
- 라이선스는 논문에 적혀 있지 않습니다. 쓰기 전에 Zenodo 페이지에서 확인해야 합니다.

firstCV에 적용한다면

- **선생님 파이프라인의 원본 영상으로 씁니다.** 영상을 `02_extract_frames.py` → `04_auto_label.py` 로 돌리면 보행 시점 탐지 라벨이 생깁니다. 이때 영상마다 붙은 장애물 이름이 선생님 라벨을 점검하는 힌트가 됩니다. 예를 들어 "bench" 영상인데 bench 박스가 하나도 없다면 라벨링이 실패한 것입니다.
- **앱 클래스와 겹치는 종류:** 2륜차(자전거·오토바이), 4륜차(차), 벤치, 쓰레기통, 의자, 테이블, 라바콘, 가로수, 주차 방지 기둥(볼라드)이 앱의 보행 라벨 목록과 겹칩니다.
- **도메인 다양성:** 지금 데이터는 실내 책상 장면에 치우쳐 있습니다. 밤과 흐린 날의 실외 보도 장면을 더해 AI Hub 보도 데이터(`12_aihub_sidewalk.py`)를 보완할 수 있습니다.
- **평가에 쓴다면** 반드시 영상 단위로 학습과 시험을 나눕니다.

정리 방법: 세 논문의 arXiv 원문 PDF를 직접 읽고 정리했습니다. 표의 수치는 원문 표에서 옮겼고, 소수점은 일부 반올림했습니다. "firstCV에 적용한다면" 항목과 ③의 데이터 누수 지적은 정리하면서 덧붙인 의견이며, 논문의 주장이 아닙니다. firstCV 수치는 README의 현재 배포 모델(open2_s, 10클래스 기준) 값입니다.